

Entwurf zum Thema „Sortialgorithmen“

Algorithmen und Datenstrukturen – WiSe 2026

Name	Vorname	Matrikel Nummer
Naghashi	Amirhossein	2792529
Pfeiffer	Tjark	2673550

Inhaltsverzeichnis

1. Einleitung.....	3
2. Anforderungen	3
2.1 Funktionale Anforderungen	3
2.2 Technische Anforderungen	3
3. Datenstrukturen	4
4. Detaillierte Algorithmen	4
4.1 Insertionsort	5
4.2 Radixsort	6
4.3 Heapsort	7
4.4 Quicksort.....	8
4.5 Introsort	9
5. Testplan	10
5.1 Teststrategien, Testfälle und Szenarien.....	10
5.2 Ziel der Tests, Hardware, Software und Testdaten	10
5.3 Erwartungen der Laufzeit.....	10

1. Einleitung

Diese Aufgabe befasst sich mit dem Entwurf und der Analyse von vier Sortieralgorithmen: Insertion Sort, Radixsort, Heapsort und Introsort. Ziel ist es, das grundlegende Prinzip jedes Algorithmus unabhängig von einer konkreten Programmiersprache zu beschreiben und zu begründen.

Die Umsetzung erfolgt in Erlang. Dabei gelten technische Einschränkungen: Zahlen werden ausschließlich in Erlang-Listen gespeichert, der Zugriff erfolgt nur per Pattern Matching.

Zusätzlich werden Laufzeittests mit zufällig, aufsteigend und absteigend sortierten Daten durchgeführt, um die Algorithmen unter verschiedenen Bedingungen zu vergleichen und ihre Leistungsfähigkeit zu bewerten.

2. Anforderungen

Das System darf ausschließlich selbst implementierte Datenstrukturen (mit Ausnahme von Listen) und Kontrollstrukturen verwenden. Die Nutzung vordefinierter Erlang-Konstrukte für diese Zwecke ist nicht zulässig.

2.1 Funktionale Anforderungen

- Insertion Sort — Schnittstelle: `insertionS:insertionS(<Liste>)`
- Radix Sort — Schnittstelle: `radixS:radixS(<Liste>, <Digit>)` — Digit = maximale Stellenzahl
- Introsort — Schnittstelle: `introS:introS(<pivot-methode>, <Liste>, <switch>)` — Pivot: left/middle/right/median/random; Wechsel zu InsertionSort bei ≤ 16 Elementen; Wechsel zu HeapSort wenn Rekursionstiefe $> \text{switch} \times \log_2(n)$
- Heap Sort — Schnittstelle: `heapS:heapS(<Liste>)`
- ADT Heap (`heap.erl`) — Schnittstellen: `create/0`, `isEmpty/1`, `insert/2`, `top/1`, `pop/1`

2.2 Technische Anforderungen

Insertion Sort — Schnittstelle: `insertionS:insertionS(<Liste>)`

Radix Sort — Schnittstelle: `radixS:radixS(<Liste>, <Digit>)` — Digit = maximale Stellenzahl

Introsort — Schnittstelle: `introS:introS(<pivot-methode>, <Liste>, <switch>)` — Pivot: left/middle/right/median/random; Wechsel zu InsertionSort bei ≤ 16 Elementen; Wechsel zu HeapSort wenn Rekursionstiefe $> \text{switch} \times \log_2(n)$

Heap Sort — Schnittstelle: `heapS:heapS(<Liste>)`

ADT Heap (`heap.erl`) — Schnittstellen: `create/0`, `isEmpty/1`, `insert/2`, `top/1`, `pop/1`

3. Datenstrukturen

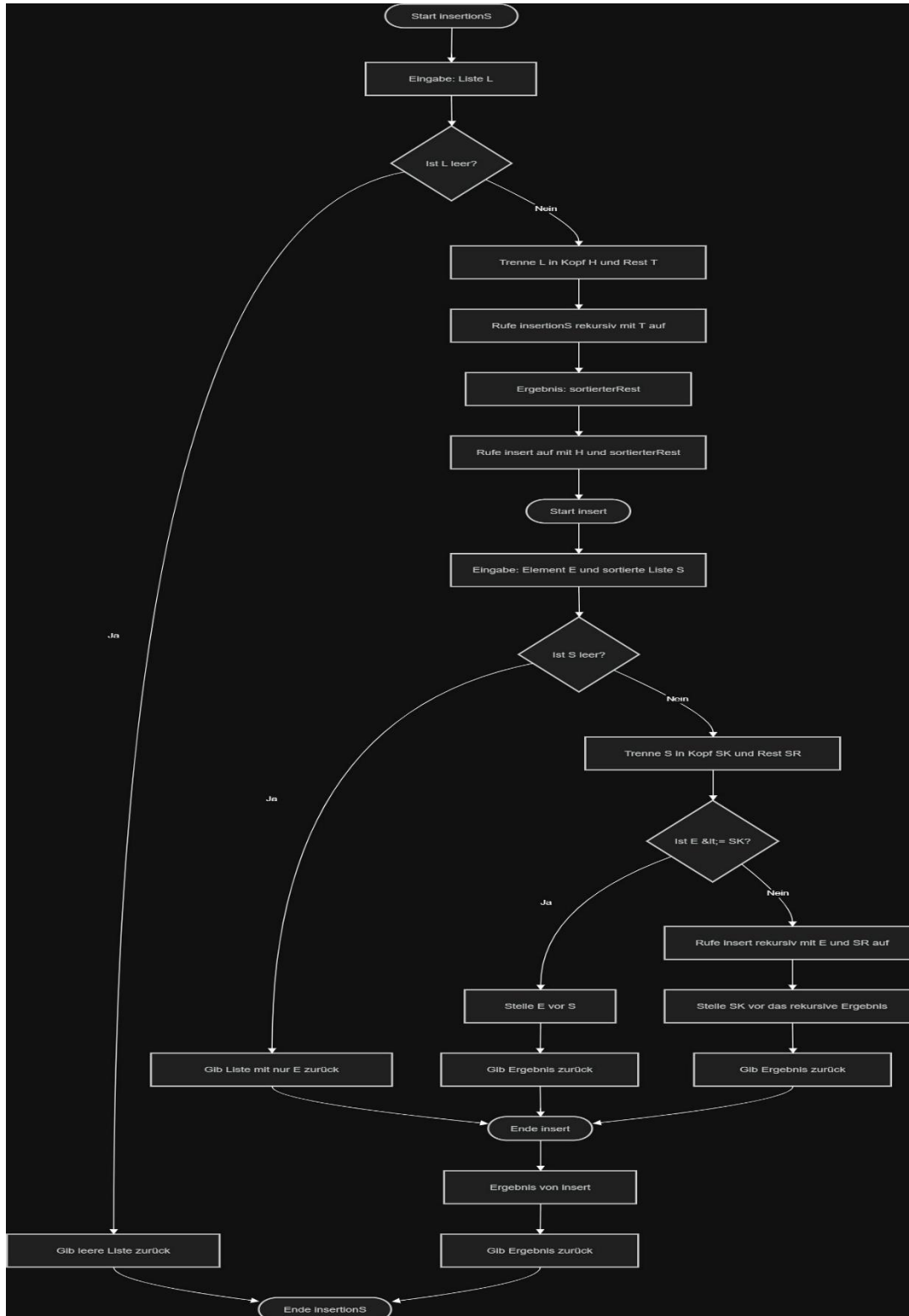
- a) **Erlang-Liste** [] Die Erlang-Liste dient als zentrales Ein- und Ausgabeformat für sämtliche Sortieralgorithmen. Der Zugriff auf Listenelemente erfolgt ausschließlich über Pattern Matching in der Form `[Head | Tail]`. Diese Struktur wird von Erlang bereitgestellt und ist gemäß Aufgabenstellung als einzige vordefinierte Datenstruktur zulässig.
- b) **Max-Heap (ADT)** Der Max-Heap wird als eigene abstrakte Datenstruktur in der Datei `heap.erl` realisiert und intern durch das Tupel `{Tree, Pos}` repräsentiert. Dabei bezeichnet `Tree` den zugrundeliegenden Binärbaum und `Pos` die nächste freie Einfügeposition, beginnend bei 1. Die Knoten sind von oben nach unten und von links nach rechts nummeriert: Knoten i besitzt das linke Kind $2i$, das rechte Kind $2i+1$ sowie den Vorgänger $\lfloor i/2 \rfloor$. Der Heap ist zu jeder Zeit kompakt zu halten, d.h. alle Ebenen außer der untersten sind vollständig besetzt, und die unterste Ebene wird von links nach rechts befüllt. Die Heap-Eigenschaft besagt, dass jeder Knoten größer oder gleich seinen Nachfolgern ist (Max-Heap-Bedingung).
- c) **Buckets (Radix Sort)** Für den Radix-Sort-Algorithmus werden zehn geordnete Fächer (Buckets) verwendet, eines für jede Dezimalziffer von 0 bis 9. Jedes Fach wird als Liste realisiert. In der Partitionierungsphase werden die Elemente anhand der aktuell betrachteten Stelle ihrer Schlüssel den entsprechenden Fächern zugewiesen. In der anschließenden Sammelphase werden die Fächer in aufsteigender Reihenfolge (0 bis 9) wieder zu einer Liste zusammengeführt, wobei die Reihenfolge der Elemente innerhalb eines Fachs erhalten bleibt. Dieser Vorgang wird beginnend bei der niedrigstwertigen Stelle (LSD) bis zur höchstwertigen Stelle wiederholt.

4. Detaillierte Algorithmen

In den folgenden Abschnitten werden die jeweiligen Algorithmen mit einer Abbildung illustriert und abschnittsweise erklärt:

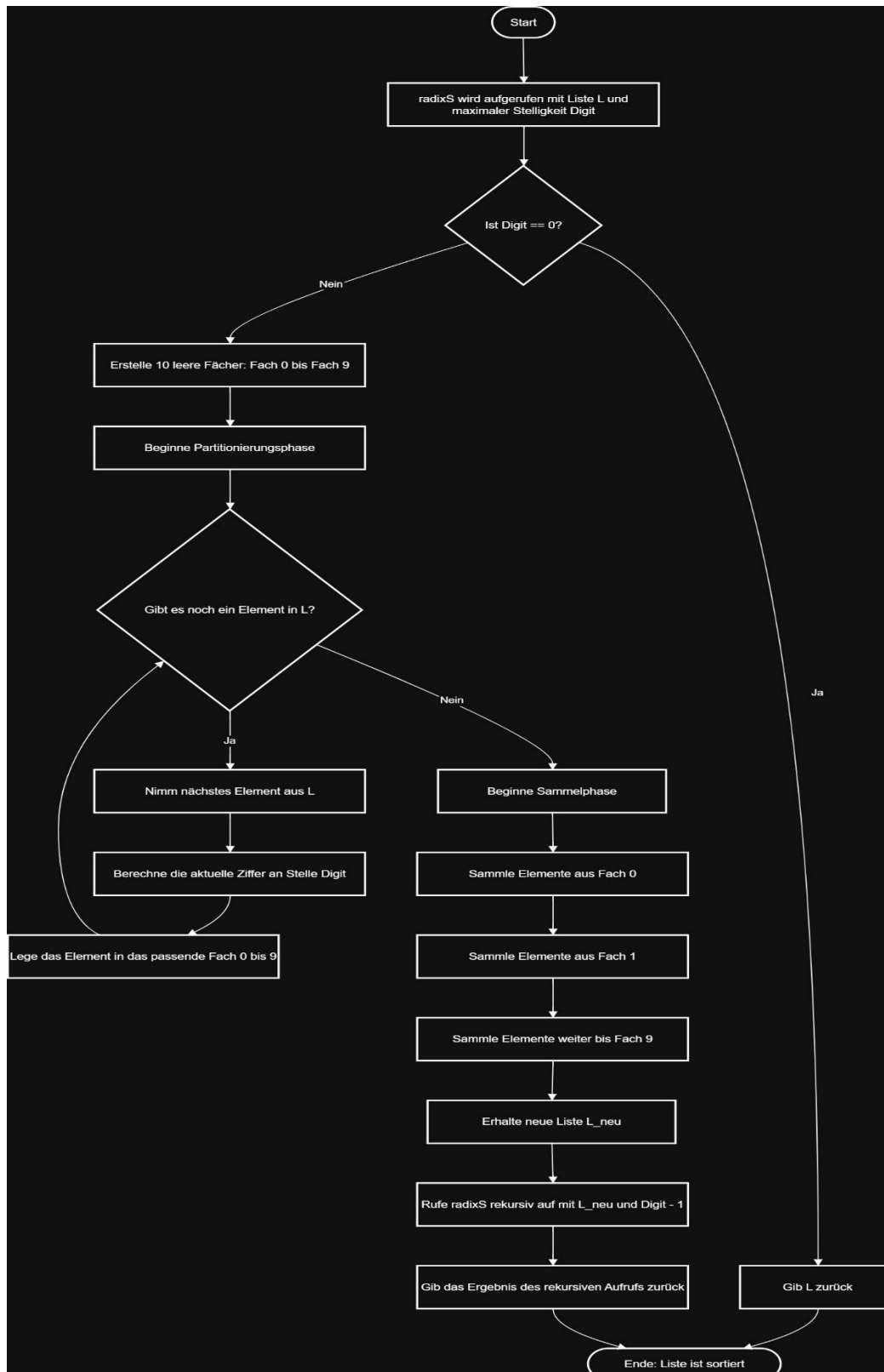
4.1 Insertionsort

Insertion Sort wird durch zwei Listen realisiert (siehe Abb.X). Die unsortierte und eine sortierte Liste. Jedes Element aus der unsortierten Liste $\langle L \rangle$ wird an der richtigen Stelle in die sortierte Liste eingefügt.



4.2 Radixsort

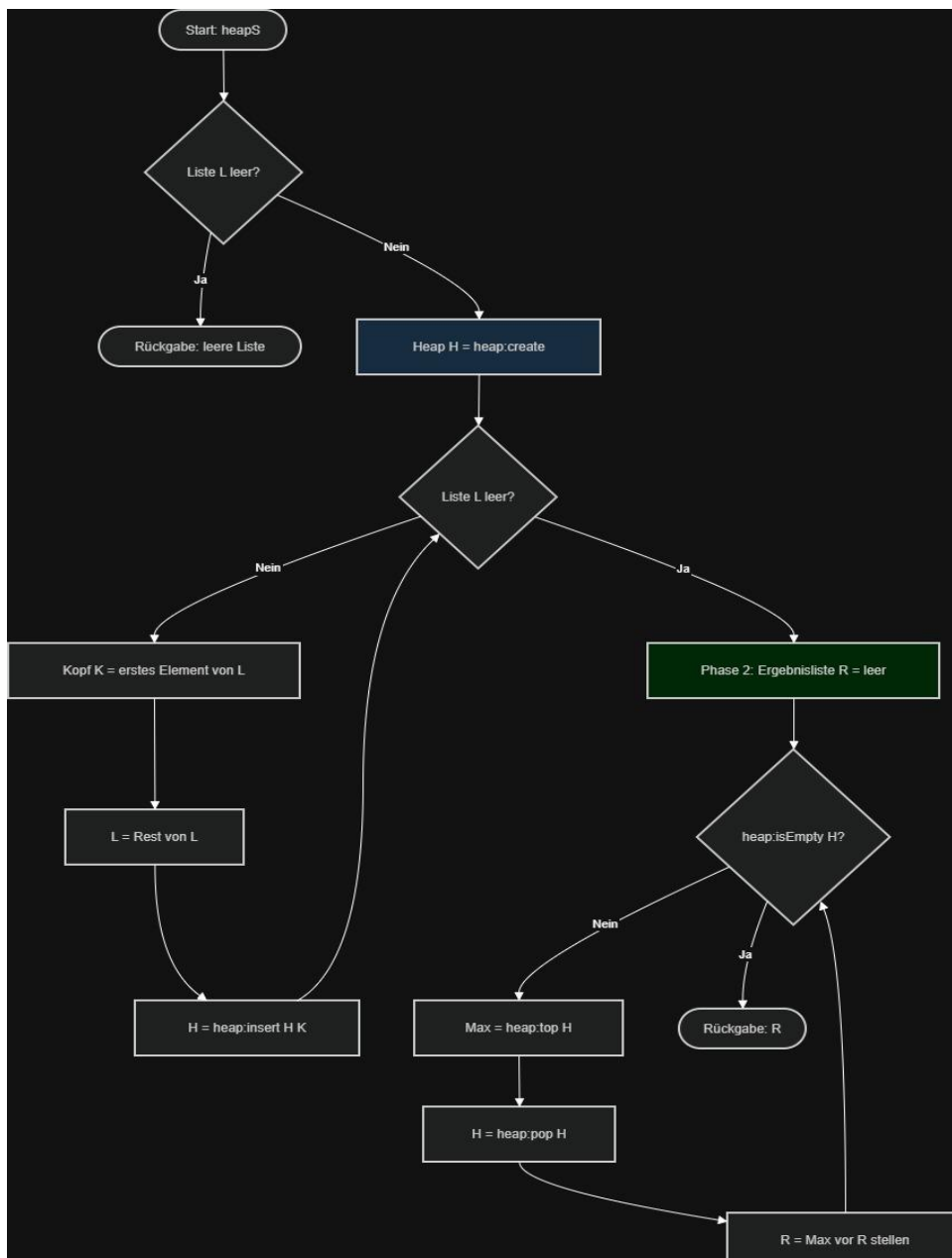
Radixsort wird durch eine einzelne Liste realisiert, die in mehreren rekursiven Aufrufen sortiert wird. Bei jedem Aufruf wird die Liste anhand des Wertes jeden Elements an einem Index sortiert. Dieser Index steigt mit jedem Durchlauf.



4.3 Heapsort

Heap Sort nutzt einen Max-Heap als zentrale Hilfsstruktur und arbeitet in zwei Phasen. In Phase 1 werden die Elemente der unsortierten Liste einzeln in einen anfangs leeren Max-Heap eingefügt (reHeap_up). Nach jedem Einfügen wird das neu hinzugefügte Element mit seinem Vorgängerknoten verglichen und bei Bedarf nach oben getauscht, bis die Heap-Eigenschaft wiederhergestellt ist – die Wurzel enthält also stets das größte Element. Phase 1 hat eine Gesamtlaufzeit von $O(n)$. In Phase 2 wird das Wurzelement (das Maximum) wiederholt aus dem Heap entnommen und in die Ergebnisliste übernommen (reHeap_down). Nach jeder Entnahme rückt das letzte Element an die Wurzel und wird nach unten bis zur richtigen Position getauscht. Phase 2 wird n -mal ausgeführt, jeder Schritt kostet $O(\log n)$, woraus eine Gesamtlaufzeit von $O(n \log n)$ resultiert.

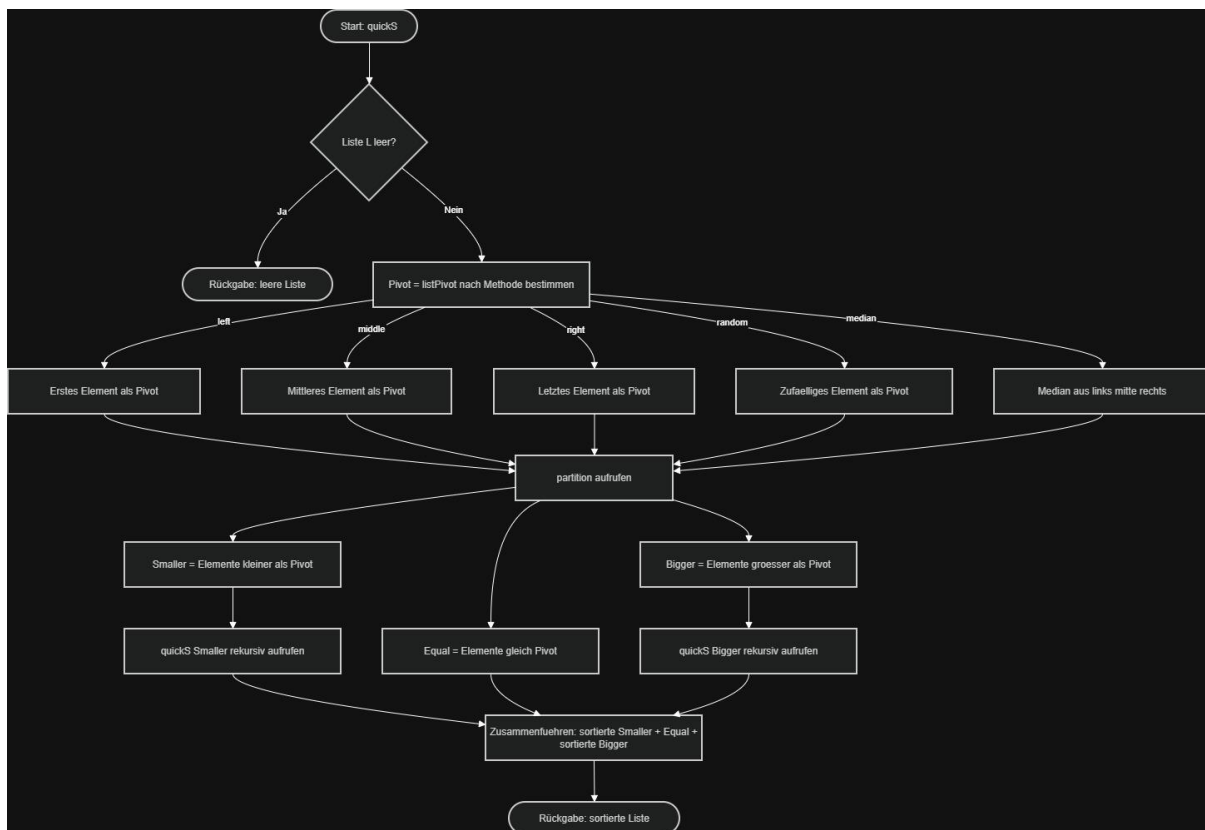
Die Kompaktheit des Heaps muss dabei stets gewährleistet sein: Alle Ebenen außer der untersten sind vollständig besetzt, und die unterste Ebene wird von links nach rechts befüllt.



4.4 Quicksort

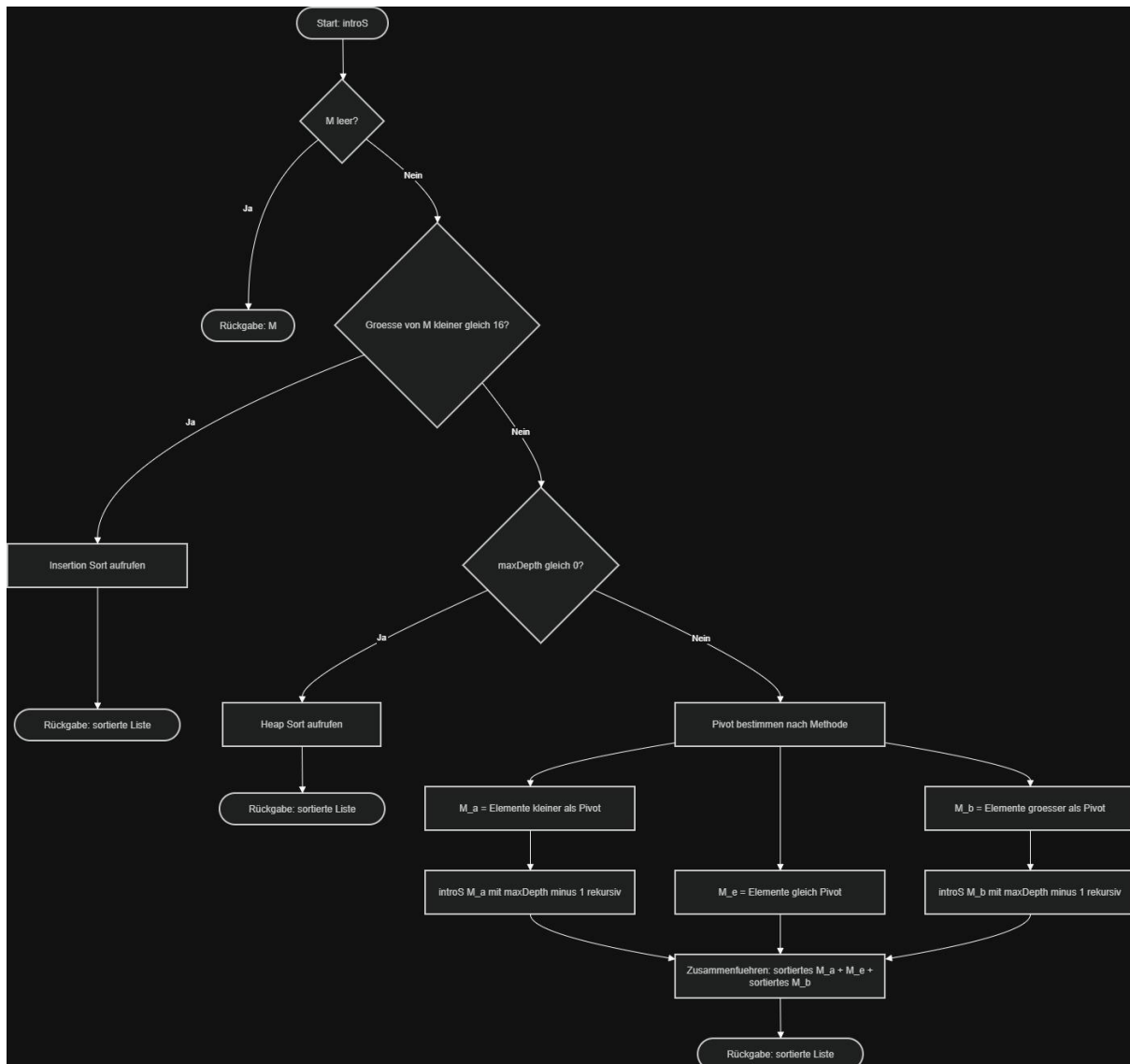
Quicksort realisiert das Teile-und-Herrsche-Prinzip durch eine dreifache Partitionierung. In jedem rekursiven Schritt wird zunächst ein Pivot-Element aus der aktuellen Liste bestimmt. Dies geschieht über die Funktion `listPivot`, die fünf verschiedene Auswahlstrategien unterstützt: das linkeste Element (`left`), das mittlere Element (`middle`), das rechteste Element (`right`), den Median der drei genannten Elemente (`median`) sowie ein zufällig gewähltes Element (`random`).

Anschließend zerlegt die Funktion `partition` die Liste in einem einzigen Durchlauf in drei disjunkte Teillisten: `Smaller` enthält alle Elemente kleiner als der Pivot, `Equal` alle Elemente gleich dem Pivot und `Bigger` alle Elemente größer als der Pivot. Da die `Equal`-Liste bereits an ihrer endgültigen Position steht, wird nur `Smaller` und `Bigger` rekursiv weiter sortiert. Die Teilergebnisse werden abschließend zu einer sortierten Gesamtliste zusammengesetzt.



4.5 Introsort

Introsort realisiert einen adaptiven Sortieralgorithmus, der je nach Situation zwischen drei Verfahren wechselt. Grundsätzlich wird die Liste durch Quicksort in die Teillisten M_a (kleiner als Pivot) und M_b (größer als Pivot) partitioniert und rekursiv verarbeitet. Zwei Grenzwerte steuern dabei den Wechsel zu effizienteren Verfahren: Überschreitet die Rekursionstiefe den Schwellwert ($\text{maxDepth} == 0$), wird Heap Sort eingesetzt, um das Worst-Case-Verhalten von Quicksort zu vermeiden. Enthält eine Teilliste hingegen 16 oder weniger Elemente, übernimmt Insertion Sort, da es bei kleinen Listen effizienter arbeitet. Eine leere Liste wird als Sonderfall sofort ohne weitere Verarbeitung zurückgegeben.



5. Testplan

Ziel dieses Abschnitts ist es, die Teststrategie vollständig zu beschreiben: von den allgemeinen Szenarien über die konkrete Testdurchführung bis hin zu den erwarteten Laufzeitergebnissen.

5.1 Teststrategien, Testfälle und Szenarien

Alle Algorithmen werden unter folgenden Grundszenarien getestet:

- Leere Eingabeliste: Jeder Algorithmus muss eine leere Liste erkennen und ohne weiteren Aufwand zurückgeben.
- Sortierte und teilsortierte Listen: Der Einfluss des Vorsortierungsgrades auf die Laufzeit wird explizit untersucht.
- Zufällige Listen: Der praxisnahe Standardfall ohne jegliche Vorsortierung.

Insertion Sort: Laufzeitmessungen mit Listen verschiedener Größen bilden die Grundlage zur Bewertung des quadratischen Wachstumsverhaltens.

Quick Sort: Der gewählte 3-Bucket-Ansatz partitioniert die Liste in Smaller, Equal und Bigger. Da die Equal-Liste keine weitere Verarbeitung benötigt, erzielt dieser Ansatz bei Eingaben mit vielen Duplikaten einen klaren Laufzeitvorteil gegenüber einer klassischen 2-Wege-Partitionierung.

Introsort: Getestet werden alle Pivot-Strategien sowie verschiedene Schwellwerte für den Wechsel zu Heap Sort. Ziel ist es, die optimale Konfiguration für unterschiedliche Eingabetypen zu ermitteln.

Heap Sort: Die Laufzeitmessung wird in Heap-Aufbau und eigentliche Sortierung unterteilt, um den relativen Anteil beider Phasen beurteilen zu können.

Radix Sort: Messungen über verschiedene Listengrößen und Ziffernlängen erlauben eine Einschätzung der linearen Laufzeitcharakteristik.

5.2 Ziel der Tests, Hardware, Software und Testdaten

Ziel der Tests: Die Tests sollen zeigen, unter welchen Eingabebedingungen welcher Algorithmus die geringste Laufzeit aufweist, um eine fundierte Empfehlung für den jeweiligen Anwendungsfall treffen zu können.

Hardware und Software: Alle Tests werden auf Standard-computern unter Windows und eventuell iOS durchgeführt – nativ oder in einer Virtualisierungsumgebung. Als Laufzeitumgebung dient die Windows-Version der Erlang-Shell.

Struktur der Testdaten: Die Testlisten werden zufällig generiert und in mehreren definierten Größenstufen eingesetzt. Jeder Algorithmus wird isoliert getestet. Die Laufzeit wird durch Differenzbildung zweier Zeitstempel – zu Beginn und am Ende der Ausführung – in Millisekunden erfasst.

5.3 Erwartungen der Laufzeit

Insertion Sort: Im Worst-Case ist quadratisches Laufzeitverhalten zu erwarten, da jedes Element im schlechtesten Fall mit allen vorherigen verglichen werden muss. Bei vorsortierten Daten reduziert sich der Aufwand erheblich, da kaum Verschiebungen erforderlich sind.

Radix Sort: Die Laufzeit ist linear und wächst proportional zur Anzahl der Elemente multipliziert mit der maximalen Stellenzahl. Sie ist damit weitgehend unabhängig von der Werteverteilung der Eingabe.

Quick Sort: Der 3-Bucket-Ansatz zeigt sein volles Potenzial bei Eingaben mit hohem Duplikateanteil. In solchen Fällen wird ein erheblicher Teil der Liste direkt als Equal klassifiziert und muss nicht rekursiv weiterverarbeitet werden.

Heap Sort: Während die Initialisierungsphase bei kleinen Listen einen vergleichsweise hohen Anteil an der Gesamtlaufzeit ausmacht, wird dieser Overhead bei wachsender Listengröße zunehmend vernachlässigbar. Die asymptotische Laufzeit beträgt $O(n \log n)$ und bleibt unabhängig von der Eingabeverteilung stabil.

Introsort: Die Laufzeit hängt stark von der gewählten Pivot-Strategie ab. Während deterministische Strategien wie links, rechts oder mitte konstante Auswahlkosten haben, kann die zufällige Wahl zu größerer Streuung führen. Die Median-Strategie liefert erfahrungsgemäß die stabilsten Ergebnisse, ist jedoch mit etwas höherem Auswahlaufwand verbunden.

Allgemein: Alle Vergleichsmessungen sind unter identischen Bedingungen durchzuführen: gleicher Rechner, gleiche Erlang-Instanz, gleiche Testdaten, minimale Hintergrundlast. Nur so lassen sich die Laufzeitunterschiede zwischen den Algorithmen zuverlässig interpretieren und gegenüberstellen.